# Amazon CodeGuru

Explore AWS CodeGuru's capabilities to improve your code quality and application performance. Understand how the Reviewer automates code analysis for Java and Python, identifying security vulnerabilities and inefficiencies, while the Profiler provides runtime insights to optimize live applications. Gain practical knowledge on integration with repositories and how to leverage CodeGuru in your CI/CD pipeline for enhanced development efficiency.

AWS CodeGuru is a machine learning-powered service from Amazon Web Services (AWS) designed to help developers improve code quality, identify security vulnerabilities, and optimize application performance. It's essentially an intelligent assistant for our code.

CodeGuru has two main components: CodeGuru Reviewer and CodeGuru Profiler. Let's discuss them in detail.

## CodeGuru Reviewer

AWS CodeGuru Reviewer is a machine learning-powered service from Amazon Web Services (AWS) that automates code reviews to help developers improve their code quality and identify potential issues for improving their Java and Python code.

Ask

CodeGuru does not identify syntax errors, but identifies defects in code, such as:

- **Code quality issues:** It identifies inefficient code, resource leaks, concurrency issues, and deviations from best practices.
- **Security vulnerabilities:** It can find common security flaws, hardcoded secrets such as passwords or API keys directly in the code, and insecure use of AWS APIs. Additionally, it integrates with Secrets Manager and uses a secret detector to look for unprotected secrets in code.
- **Performance bottlenecks:** It identifies code patterns that are likely to cause performance problems.

## CodeGuru Reviewer integration with repositories

CodeGuru Reviewer is designed to fit seamlessly into existing development processes. It can integrate with the Python and Java repositories in Bitbucket, GitHub, GitHub Enterprise Cloud, GitHub Enterprise Server, and Amazon S3.

Once CodeGuru is enabled in the repository, we can use it to automatically analyze code changes in pull requests or merge requests and provide feedback as comments directly within the pull request, allowing developers to address issues before merging. We can also initiate full scans of an entire repository for a comprehensive analysis.

Simply put, AWS CodeGuru Reviewer acts as an intelligent, automated assistant that helps development teams write higher-quality, more secure, and more efficient code.

To understand it better, let's learn how to use CodeGuru Reviewer for a GitHub repository. First, we must set up a GitHub connection via AWS CodeConnections. Then we associate the GitHub repo with CodeGuru using the command given below:

Ask

main.sh Shell

```
aws codeguru-reviewer associate-repository \
  --repository 'ThirdPartySourceRepository={Name=my-repo,ConnectionAr
```

Command to associate GitHub repository with CodeGuru reviewer

With this setup, we can seamlessly integrate CodeGuru reviewer into our CI/CD pipeline as shown in the illustration below. The developers, working on their separate IDEs, commit code to a repository, which triggers CodeGuru reviewers. After the developers address the recommendations by the reviewer in the code, it can be deployed to production via a comple CI/CD pipeline.

Ask

CodeGuru reviewer's integration in CI/CD environment

## CodeGuru Profiler

AWS CodeGuru Profiler is a machine learning-powered service that helps developers understand and optimize the runtime performance of live applications. It continuously collects data from the running applications in production environments, such as those on EC2, Lambda, ECS, EKS, or even on-premises servers.

CodeGuru analyzes various metrics like CPU utilization, memory usage, and latency patterns to identify:

- Lines of code that are consuming the most CPU.
- Inefficient loops or algorithms.

Ask

- Memory leaks or excessive memory allocation.
- Contention issues that are slowing down processes.
- Issues related to I/O operations or database calls.

CodeGuru supports applications written in Java Virtual Machine (JVM) runtimes and Python 3.6 and later. It creates a profiler that collects runtimes and organizes application data in a profiling group.

We can create a profiling group using a simple command given below:

main.sh Shell

```shell
aws codeguruprofiler create-profiling-group \
  --profiling-group-name my-profiling-group \
  --compute-platform AWSLambda
```

Command to create a profiling group

Notice the compute-platform argument. We select `AWSLambda` as compute platform for our profiling group if our applications runs on Lambda. For applications hosted on any other compute services such as EC2 or on premesis services, we use `Default` compute platform.

Ask

## Visualizations and recommendations

CodeGuru supports visualizations of profiling data collected by the profiling group.

### Types of visualizations

CodeGuru Profiler offers three main types of visualizations. Each serves a distinct purpose in helping us investigate performance issues.

- **Inspect:** This visualization allows us to explore the call stacks in detail, focusing on a function's specific execution path. It's like magnifying a particular segment of our code's execution to see which sub-functions it calls and how much time each contributes.
- **Overview:** This is often presented as a Flame Graph, providing a "bottom-up" view of our application's resource consumption for exmaple CPU, Latency, or Heap. The width of a stack frame indicates the total time that a function and its children spent in a specific state, making it excellent for quickly spotting broad areas of high resource usage.
- **Hotspots:** This visualization offers a "top-down" perspective, highlighting the functions that are directly consuming the most application time. It prioritizes the functions that are the biggest contributors to our application's slowness or resource drain, regardless of where they appear in the call stack.

### Accessing visualizations

CodeGuru allows us to select any profiling group and offers three main types of visualizations:

- **Visualize CPU:** It graphically displays how much of the CPU's processing power is being used over time, helping identify if it's a performance bottleneck. This includes showing total usage, per-core activity, and idle time.
- **Visualize Latency:** It shows the delay or time taken for operations to complete, like network requests or database queries. It's crucial for

Ask

understanding responsiveness and pinpointing slow parts of a system.

- **Visualize Heap:** This utility provides a visual representation of an application's dynamic memory usage, the heap. It helps detect memory leaks, analyze object allocation patterns, and optimize garbage collection.

Apart from the visualization, CodeGuru provides machine learning-driven recommendations on how to fix the identified problem, along with an estimate of the cost saved.

Ask